SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4307
STUDENT NAME: K. Ganesh
REGISTER NUMBER: 192411170

COURSE OUTCOME COVERED

CO -2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Design Task

Weightage: 20%

QUESTIONS: **Total Marks: 20**

Question 1: Responsive Navigation Bar Design - Marks(4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.
(a) Identify key components required in a navigation bar for an e-commerce website.
(b) Design a responsive navigation bar using appropriate CSS techniques.
(c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).
(d) Evaluate the usability and suggest improvements for better user experience.

Question 2: Product Hover Effect - Marks(4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.
(a) Identify suitable CSS properties for hover effects.
(b) Suggest appropriate **CSS animation or transition** for product interaction.
(c) Design a hover effect for a product card.
(d) Evaluate performance and user experience considerations.

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.
(a) Identify layout requirements of a dashboard (rows, columns, alignment).
(b) Compare **Flexbox and Grid** based on layout needs.
(c) Choose the most suitable layout technique and justify your choice.
(d) Evaluate the scalability and responsiveness of the chosen approach.

Question 4: Mobile-First Blog Layout - Marks(4)

Suppose that a blog page must be designed following a mobile-first approach.
(a) Identify key sections of a blog layout.
(b) Design a mobile-first layout using CSS.
(c) Demonstrate how the layout scales for larger screens using media queries.
(d) Evaluate accessibility and readability of the design.

Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

Suppose that a website includes a dynamic menu that responds to user interactions.
(a) Identify appropriate JavaScript events for menu interaction.
(b) Design event handling logic for menu toggle functionality.
(c) Implement interaction for user actions (click/hover).
(d) Evaluate performance and suggest improvements.

Code of Conduct:

I K. Ganesh / 192411170 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: *K. Ganesh*

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete understanding of design	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts

		requirements and concepts			
Design / Implementation	30	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Question 1: Responsive Navigation Bar Design

(a) Identify key components required in a navigation bar for an e-commerce website.

- **Logo or brand name** that links back to the home page.
- **Main links:** Home, Shop (categories), Deals and Contact.
- **Search bar**, because shoppers usually look for a product by name.
- **Cart icon with an item-count badge**, plus an **Account / Login** link (and wishlist if needed).
- **Menu toggle (hamburger)** that appears only on small screens.

(b) Design a responsive navigation bar using appropriate CSS techniques.

I used Flexbox with **`flex-wrap`** and the **`order`** property, written mobile-first: the base CSS is for phones, and media queries add the tablet and desktop layouts.

```
<header class="site-header">
  <a href="/" class="logo">ShopEasy</a>
  <button class="menu-toggle" aria-label="Menu"
          aria-expanded="false" aria-controls="main-nav">&#9776;</button>
  <nav id="main-nav" class="main-nav">
    <ul>
      <li><a href="/">Home</a></li>
      <li><a href="/shop">Shop</a></li>
      <li><a href="/deals">Deals</a></li>
      <li><a href="/contact">Contact</a></li>
    </ul>
  </nav>
  <form class="search" role="search">
    <input type="search" placeholder="Search products"
           aria-label="Search products">
  </form>
  <a href="/cart" class="cart">Cart <span class="badge">3</span></a>
</header>
```

```
/* mobile first (base styles) */
.site-header { display: flex; flex-wrap: wrap; align-items: center;
  gap: 10px; padding: 10px 16px; background: #fff;
  border-bottom: 1px solid #999; position: sticky; top: 0; }
.logo        { order: 1; font-weight: bold; margin-right: auto; }
.cart        { order: 2; }
.menu-toggle { order: 3; min-height: 44px; min-width: 44px;
               font-size: 1.5rem; background: none; border: 0; }
.search      { order: 4; flex: 1 1 100%; }
.search input { width: 100%; padding: 10px; }
.main-nav    { order: 5; flex: 1 1 100%; display: none; }
.main-nav.open { display: block; }
.main-nav ul { list-style: none; margin: 0; padding: 0; }
.main-nav a  { display: block; padding: 12px 8px; color: #000;
               text-decoration: none; }
```

```
/* tablet and above: links in one row, no hamburger */
@media (min-width: 768px) {
  .menu-toggle { display: none; }
  .logo   { order: 1; margin-right: 0; }
  .main-nav { order: 2; flex: 0 0 auto; display: block; }
  .main-nav ul { display: flex; gap: 16px; }
  .main-nav a  { padding: 8px; }
  .search { order: 3; flex: 1 1 140px; }
  .cart   { order: 4; }
}
/* desktop: more room for the search field */
@media (min-width: 1024px) {
  .site-header { padding: 12px 40px; gap: 24px; }
  .search { flex: 0 1 380px; margin-left: auto; }
}
```

(c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).

Above 768px all links are visible in one row. Below 768px the links are hidden (**`display: none`**) and a hamburger button appears. Tapping it adds the class **open** to the nav, which shows the links as a full-width list under the header (JavaScript for this is in Question 5). Figure 1 shows the three layouts.

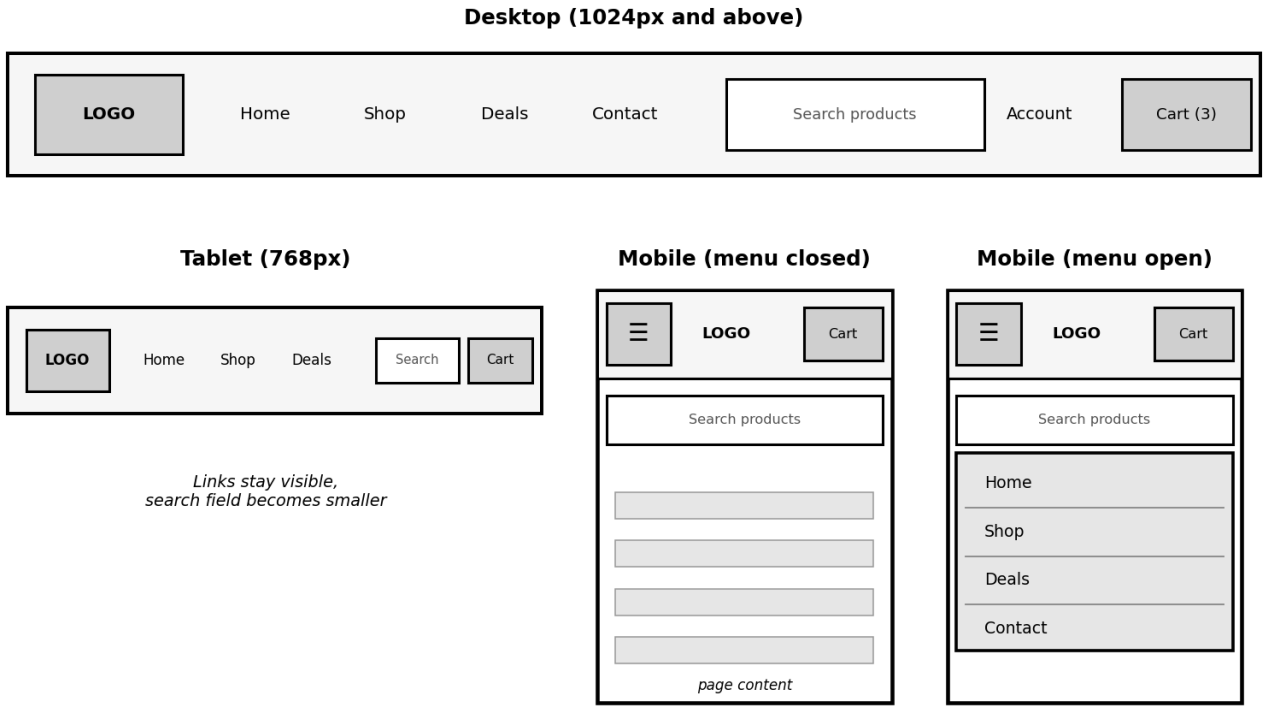


Figure 1: Navigation bar on desktop, tablet and mobile (menu closed and open)

(d) Evaluate the usability and suggest improvements for better user experience.

Good points: the search bar and cart are always visible, the header stays on top while scrolling (sticky), buttons are at least 44px high so they are easy to tap, and the links are real **`<a>`** elements, so keyboard users can reach them.

Improvements:

- Close the menu with the Esc key or by tapping outside it, and move focus back to the hamburger button.
- Add a mega-menu or dropdown for product categories, and search suggestions as the user types.
- Add a "skip to content" link, check colour contrast, and show the current page with **`aria-current="page"`**.
- On phones, consider a bottom navigation bar for the most-used actions (Home, Search, Cart, Account) because it is reachable with the thumb.

> **Conclusion:** A mobile-first Flexbox header with two breakpoints gives a menu that adapts to every screen, and the suggested improvements make it faster and easier to use.

Question 2: Product Hover Effect

(a) Identify suitable CSS properties for hover effects.

- **`transform`** (**`scale()`**, **`translateY()`**) to lift or zoom an item.
- **`box-shadow`** to give depth, **`opacity`** to fade items in or out, and **`filter`** (for example brightness).
- **`background-color`**, **`color`** and **`border-color`** for simple colour changes, and **`cursor: pointer`** to show that the item is clickable.
- **`transition`** (and **`animation`**) to make the change smooth instead of sudden.

(b) Suggest appropriate CSS animation or transition for product interaction.

A **transition** is best for hover, because it only needs a start state and an end state. I suggest **`transition: transform 0.3s ease, box-shadow 0.3s ease;`**. A **keyframe animation** can be used for attention effects, for example a short pulse on the cart badge when an item is added:

```
@keyframes pulse {
  0%   { transform: scale(1); }
  50%  { transform: scale(1.25); }
  100% { transform: scale(1); }
}
.badge.added { animation: pulse 0.4s ease; }
```

(c) Design a hover effect for a product card.

```
<article class="product-card">
  <div class="img-wrap"><img src="shoe.jpg" alt="Running shoe"></div>
  <h3>Running Shoe</h3>
  <p class="price">Rs. 2,499</p>
  <button class="add">Add to cart</button>
</article>
```

```
.product-card { width: 240px; padding: 12px; background: #fff;
  border: 1px solid #999; border-radius: 8px;
  transition: transform 0.3s ease, box-shadow 0.3s ease; }
.img-wrap { overflow: hidden; border-radius: 6px; }
.img-wrap img { width: 100%; display: block;
  transition: transform 0.4s ease; }
.add { opacity: 0; transform: translateY(8px);
  transition: opacity 0.3s ease, transform 0.3s ease; }
.product-card:hover, .product-card:focus-within {
  transform: translateY(-6px); box-shadow: 0 10px 20px rgba(0,0,0,.25); }
.product-card:hover img { transform: scale(1.08); }
.product-card:hover .add, .product-card:focus-within .add {
  opacity: 1; transform: none; }
/* touch screens have no hover: keep the button visible */
@media (hover: none) { .add { opacity: 1; transform: none; } }
/* respect users who prefer less motion */
@media (prefers-reduced-motion: reduce) {
  .product-card, .img-wrap img, .add { transition: none; }
}
```

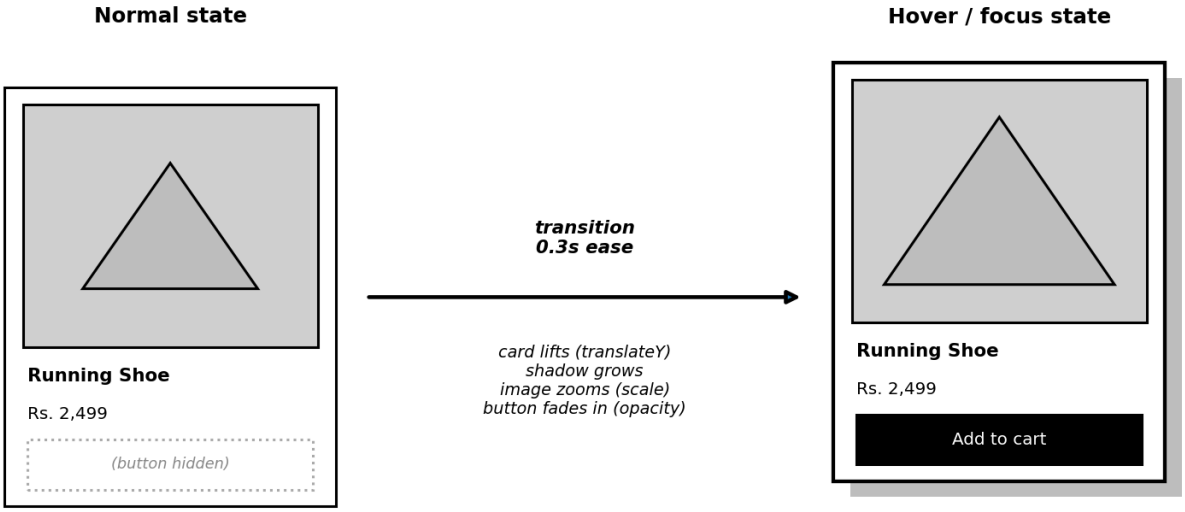


Figure 2: Product card in normal state and in hover or focus state

(d) Evaluate performance and user experience considerations.

- **Performance:** animate only **`transform`** and **`opacity`**. The browser can handle them on the GPU without recalculating the layout. Animating **`width`**, **`height`**, **`margin`** or **`top`** forces reflow and can make the page stutter.
- Avoid **`transition: all`**, and use **`will-change`** only when really needed, because it uses extra memory.
- **UX:** keep durations between about 200 and 400 ms so the effect feels quick. Do not hide important information (price, name) behind hover.
- **Touch devices** have no hover, so I used **`:focus-within`** for keyboard users and **`@media (hover: none)`** for phones. I also respect **`prefers-reduced-motion`** for people sensitive to movement.

> **Conclusion:** A short transition on transform, shadow and opacity gives a clear, smooth hover effect at very little cost, as long as it also works for keyboard, touch and reduced-motion users.

Question 3: Layout Selection (Flexbox vs Grid)

(a) Identify layout requirements of a dashboard (rows, columns, alignment).

- **Rows:** a header row, a middle row for content, and a footer row.
- **Columns:** a fixed-width sidebar (about 220px) and a flexible main area that takes the remaining space.
- **Inside the main area:** a row of equal stat cards, then a large chart next to a narrower table (2:1 ratio).
- **Alignment:** equal gaps, cards aligned to the same edges, text and icons centred inside cards, and extra widgets that can be added later without breaking the layout.
- **Responsive:** on small screens everything should stack into one column.

(b) Compare Flexbox and Grid based on layout needs.

Point	Flexbox	CSS Grid
Dimensions	One-dimensional (a row or a column at a time)	Two-dimensional (rows and columns together)
Approach	Content-first: item sizes shape the layout	Layout-first: the grid is defined, then items are placed
Best for	Navbars, toolbars, card rows, centring, components	Full page layouts, dashboards, galleries
Placing items	Order and alignment along one axis	Exact cell, span and named areas (`grid-template-areas`)
Alignment	Strong along the main and cross axis	Strong in both directions, plus **`gap`**
Wrapping	Wraps lines, but lines do not align with each other	Rows and columns stay aligned

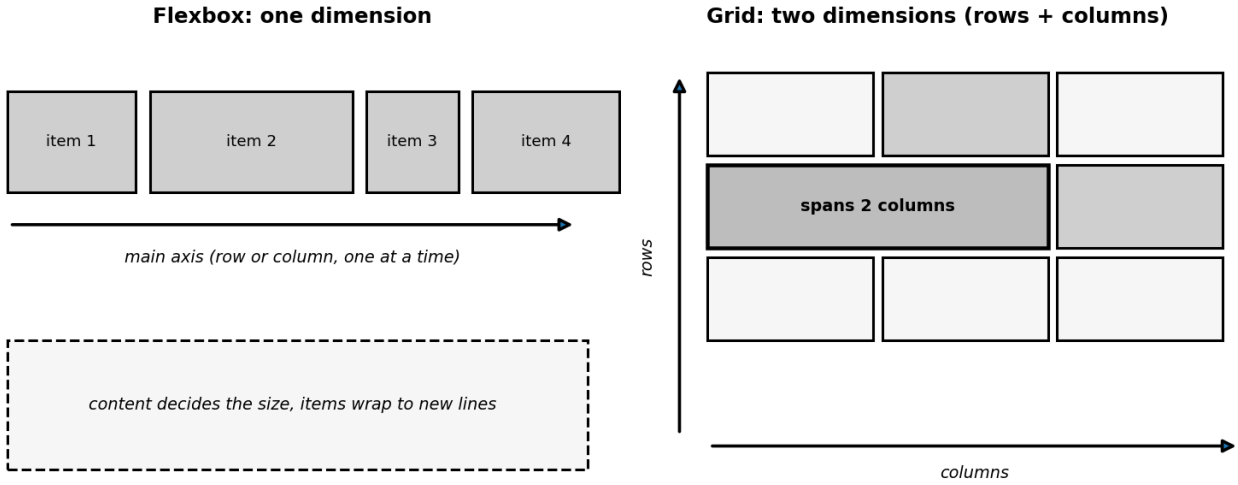


Figure 3: Flexbox works along one axis, Grid works with rows and columns

(c) Choose the most suitable layout technique and justify your choice.

I choose **CSS Grid for the page layout** of the dashboard and Flexbox inside small components. A dashboard is a two-dimensional layout: the sidebar, header, cards, chart and table must line up in rows and columns at the same time, and Grid does this directly with named areas. Doing the same with Flexbox needs nested containers and fixed widths, which is harder to maintain. Flexbox is still used for the header content and for centring items inside cards.

```
.dashboard { display: grid; min-height: 100vh;
  grid-template-columns: 220px 1fr;
  grid-template-rows: 64px 1fr 48px;
  grid-template-areas:
    "header  header"
    "sidebar main"
    "footer  footer"; }
.header  { grid-area: header; display: flex; align-items: center;
           justify-content: space-between; }
.sidebar { grid-area: sidebar; }
.main    { grid-area: main; padding: 16px; }
.footer  { grid-area: footer; }
.stats  { display: grid; gap: 16px;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); }
.panels { display: grid; gap: 16px; margin-top: 16px;
  grid-template-columns: 2fr 1fr; }
@media (max-width: 768px) {
  .dashboard { grid-template-columns: 1fr; grid-template-rows: auto;
    grid-template-areas: "header" "sidebar" "main" "footer"; }
  .panels { grid-template-columns: 1fr; }
}
```

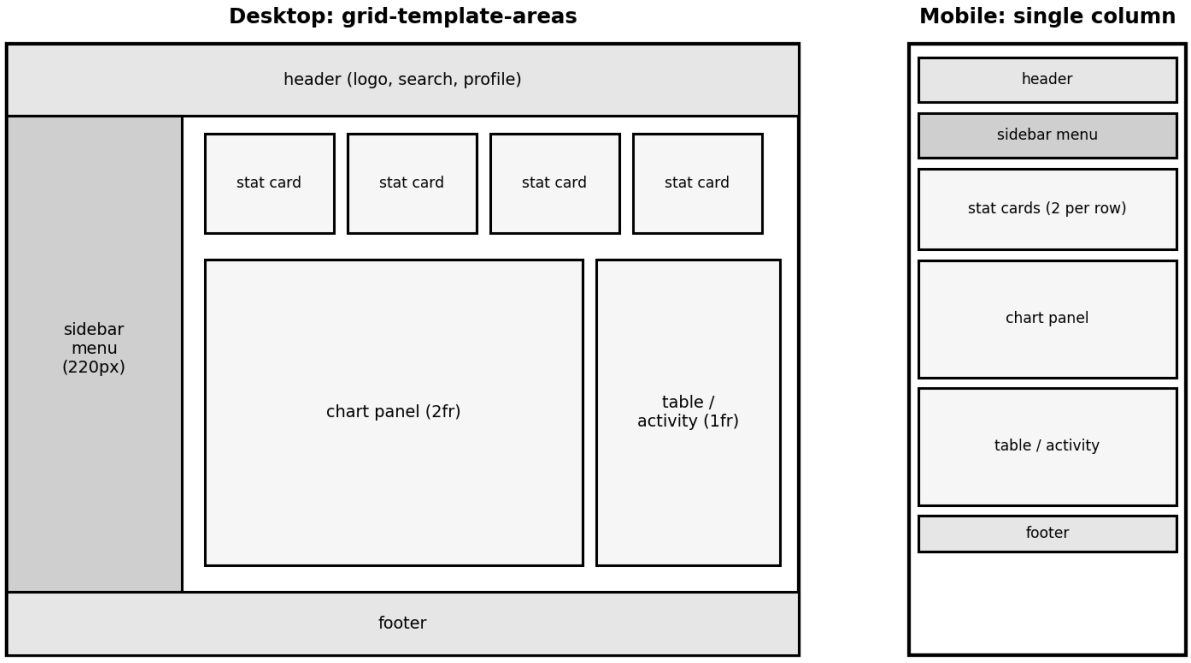


Figure 4: Dashboard layout on desktop and the stacked layout on mobile

(d) Evaluate the scalability and responsiveness of the chosen approach.

- **Scalability:** a new widget is just a new grid item. Because **`repeat(auto-fit, minmax(200px, 1fr))`** creates the columns, extra cards are added automatically without changing the CSS.
- **Responsiveness:** **`fr`** units and **`minmax()`** adjust to the screen, and for small screens only the area map and the column list are redefined in one media query.
- **Maintainability:** the named areas make the layout easy to read, and the markup order stays the same on all screens.
- **Watch out for:** long content can overflow a grid cell, so use **`min-width: 0`** on grid items that hold tables or charts. Grid is supported by all current browsers, so no fallback is needed for modern users.

> **Conclusion:** Grid for the overall structure and Flexbox for components is the most practical combination. It scales when widgets are added and adapts to any screen size.

Question 4: Mobile-First Blog Layout

(a) Identify key sections of a blog layout.

- **Header** with the blog name and navigation.
- **Post title and meta** (author, date, reading time) and a **featured image**.
- **Article content:** paragraphs, sub-headings, images and quotes.
- **Sidebar:** search, categories, recent posts and an about box. **Comments** and **related posts** come after the article.
- **Footer** with links, social icons and a newsletter form.

(b) Design a mobile-first layout using CSS.

Mobile-first means the base CSS targets the smallest screen: one column, full-width images and comfortable text. Larger screens are handled later with **`min-width`** media queries that only add what changes.

```
<header class="site-header">...</header>
<div class="layout">
  <main>
    <article class="post">
      <h1>How to Style a Responsive Page</h1>
      <p class="meta">By Asha | 12 Oct 2026 | 5 min read</p>
      <img src="cover.jpg" alt="Laptop showing a web page">
      <p>Article text...</p>
    </article>
  </main>
  <aside class="sidebar">...categories, recent posts...</aside>
</div>
<footer>...</footer>
```

```
*, *::before, *::after { box-sizing: border-box; }
body { margin: 0; font-family: Georgia, serif; font-size: 1rem;
  line-height: 1.6; color: #111; background: #fff; }
img { max-width: 100%; height: auto; }
main, .sidebar { padding: 16px; }
.post h1 { font-size: 1.6rem; line-height: 1.25; }
.meta { color: #444; font-size: 0.9rem; }
.sidebar { border-top: 2px solid #000; margin-top: 24px; }
a { color: #00008b; } a:focus { outline: 3px solid #000; outline-offset: 2px; }
```

(c) Demonstrate how the layout scales for larger screens using media queries.

```
/* tablet: centred, slightly larger text */
@media (min-width: 768px) {
  body { font-size: 1.0625rem; }
  .layout { max-width: 720px; margin: 0 auto; }
  .post h1 { font-size: 2rem; }
}
/* desktop: article and sidebar side by side */
@media (min-width: 1024px) {
  .layout { display: grid; gap: 32px; max-width: 1100px;
    grid-template-columns: minmax(0, 2fr) 1fr; }
  .post p { max-width: 70ch; }
  .sidebar { border-top: 0; margin-top: 0; }
}
```

On a phone the sidebar sits below the article. At 768px the content is centred in a readable column. At 1024px the grid places the sidebar next to the article (Figure 5).

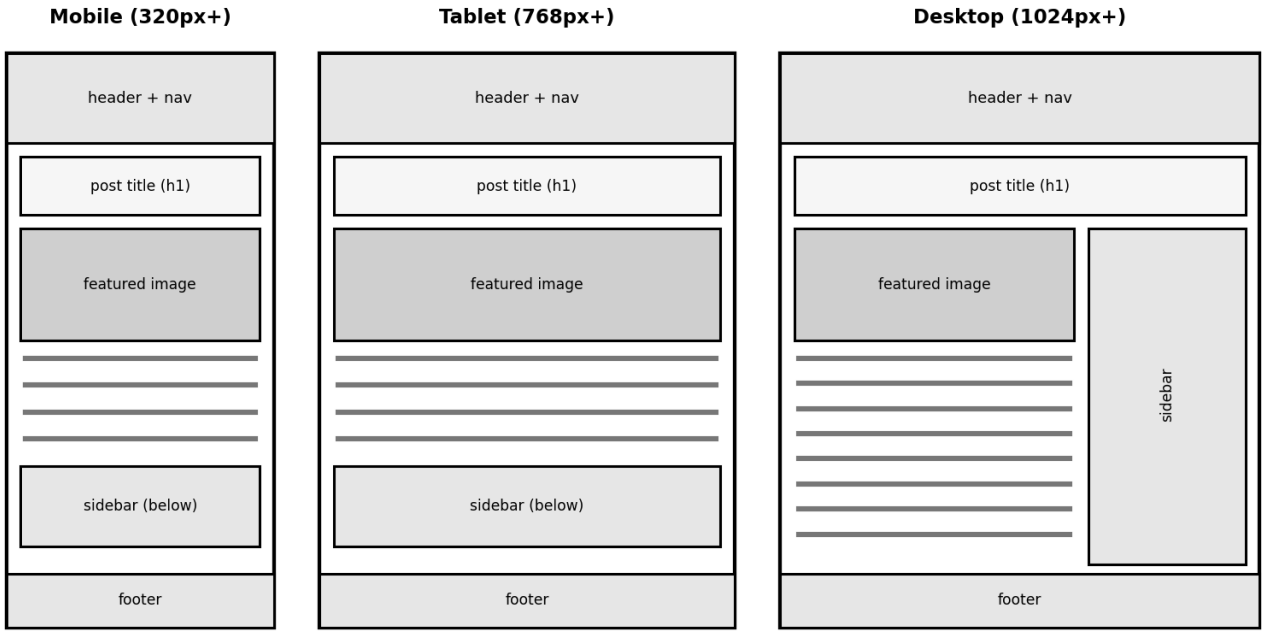


Figure 5: Blog layout at mobile, tablet and desktop widths

(d) Evaluate accessibility and readability of the design.

- **Readability:** body text is at least 16px (`1rem`), line height is 1.6, and lines are limited to about 70 characters (`max-width: 70ch`), which is comfortable to read.
- **Contrast:** dark text on a white background is well above the 4.5:1 WCAG ratio. Links are also distinguishable by colour and a visible focus outline.
- **Semantic HTML:** `<header>`, `<main>`, `<article>`, `<aside>` and `<footer>` let screen readers jump between regions, and headings follow a logical order (h1, then h2).
- **Zoom and scaling:** relative units (`rem`, `%`) let text grow when users enlarge it, and the viewport meta tag does not block zooming. Images have `alt` text.
- **Improvements:** add a skip-to-content link, a dark-mode option with `prefers-color-scheme`, and make sure tap targets are at least 44px.

> **Conclusion:** Starting from the phone layout keeps the blog fast and focused, and the media queries add columns only where there is room. Good contrast, line length and semantic tags make it readable and accessible.

Question 5: JavaScript Event Handling for Dynamic Menu

(a) Identify appropriate JavaScript events for menu interaction.

- `click`: opens or closes the menu with the hamburger button. It also works for touch and keyboard (Enter or Space on a button).
- `keydown`: the Escape key closes the menu, and arrow keys can move between items.
- `mouseenter` / `mouseleave`: show and hide a dropdown submenu on desktop with a mouse.
- `focusin` / `focusout`: do the same for keyboard users, so a submenu is not mouse-only.
- `resize` (or a `matchMedia` listener): reset the menu when the window changes between mobile and desktop.

(b) Design event handling logic for menu toggle functionality.

One function, `setMenu(open)`, does all the work: it adds or removes the class `open`, updates `aria-expanded` for assistive technology and changes the icon. Every event just calls this function.

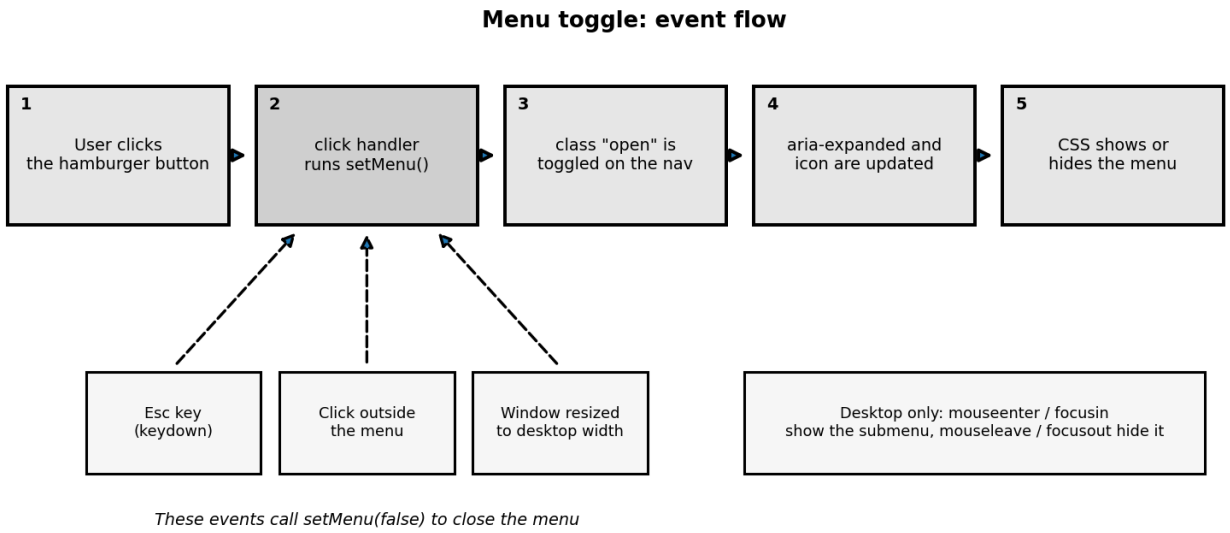


Figure 6: Event flow for the menu toggle

```
const toggle = document.querySelector(".menu-toggle");
const nav = document.getElementById("main-nav");

function setMenu(open) {
  nav.classList.toggle("open", open);
  toggle.setAttribute("aria-expanded", String(open));
  toggle.textContent = open ? "\u2715" : "\u2630";
}

toggle.addEventListener("click", () => {
  setMenu(!nav.classList.contains("open"));
});
```

(c) Implement interaction for user actions (click/hover).

Extra closing rules (Escape key, click outside) and a dropdown submenu that opens on hover for mouse users, on focus for keyboard users and on tap for touch screens:

```
document.addEventListener("keydown", (e) => {
  if (e.key === "Escape") { setMenu(false); toggle.focus(); }
});

document.addEventListener("click", (e) => {
  if (!nav.contains(e.target) && !toggle.contains(e.target)) {
    setMenu(false);
  }
});

document.querySelectorAll(".has-sub").forEach((item) => {
  const sub = item.querySelector(".submenu");
  const show = () => sub.classList.add("show");
  const hide = () => sub.classList.remove("show");
  item.addEventListener("mouseenter", show);
  item.addEventListener("mouseleave", hide);
  item.addEventListener("focusin", show);
  item.addEventListener("focusout", (e) => {
    if (!item.contains(e.relatedTarget)) hide();
  });
  item.querySelector("a").addEventListener("click", (e) => {
    if (window.matchMedia("(hover: none)").matches) {
      e.preventDefault();          /* first tap opens the submenu */
      sub.classList.toggle("show");
    }
  });
});

window.matchMedia("(min-width: 768px)").addEventListener("change",
  (e) => { if (e.matches) setMenu(false); });
```

```
.submenu { display: none; list-style: none; padding-left: 16px; }
.submenu.show { display: block; }
```

(d) Evaluate performance and suggest improvements.

- **Performance:** the handlers are tiny and only change a class, so the browser does the styling work in CSS. Using **`classList`** is better than writing inline styles in JavaScript.
- **Listeners:** a single document-level listener for Escape and outside clicks is cheaper than many separate ones. **`matchMedia`** fires only when the breakpoint is crossed, unlike **`resize`**, which fires continuously and needs debouncing.
- **Improvements:** use event delegation if the menu has many items, trap focus while the mobile menu is open, add small transitions that respect **`prefers-reduced-motion`**, and load the script with **`defer`** so it does not block rendering.
- **Accessibility:** keep **`aria-expanded`** in sync, use real **`<button>`** elements, and make sure every hover action also works with the keyboard and touch.

> **Conclusion:** A single setMenu function, driven by click, keyboard, hover and media-query events, gives a menu that is simple, fast and works for mouse, keyboard and touch users.